Decentralized Autonomous Organization(DOA) Voting System

1. Introduction

A Decentralized Autonomous Organization (DAO) Voting System is an innovative blockchain-based solution that enables transparent, secure, and democratic decision-making within decentralized communities. The system leverages smart contracts and distributed ledger technology to facilitate proposal creation, voting mechanisms, and result aggregation without requiring a centralized authority or intermediary. This ensures that governance decisions are made fairly, immutably recorded on the blockchain, and executed automatically through predefined smart contract logic.

The DAO Voting System integrates modern technologies including Ethereum smart contracts, Web3 wallet integration (MetaMask), and a React-based frontend to provide users with an intuitive interface for participating in organizational governance. The system is designed for organizations that require transparent voting mechanisms where every participant's vote is cryptographically secured and permanently recorded. This approach eliminates the possibility of vote tampering, double voting, and fraudulent activity commonly associated with traditional voting systems.

The architecture comprises multiple tiers: a Solidity smart contract layer that manages voting logic and proposal storage on the blockchain, an Express.js API server that acts as a bridge between the frontend and blockchain, a React frontend application that provides user-friendly interaction, and a Hardhat local Ethereum environment for development and testing. This multi-layered approach ensures security at every level while maintaining ease of use for end users.

From a functional perspective, the system manages the complete voting lifecycle—from proposal creation through vote submission to result calculation and display. Users connect their Ethereum wallets, authenticate via MetaMask, browse active proposals, and cast cryptographically signed votes. The system prevents double voting through on-chain state tracking and provides real-time results visualization. For administrators, the system offers proposal management capabilities and access to voting analytics.

Overall, the DAO Voting System represents a significant advancement in decentralized governance, combining the security and transparency of blockchain technology with user-friendly web interfaces. It addresses critical challenges in organizational decision-making by eliminating single points of failure, ensuring vote integrity, and enabling global participation without geographical limitations.

Ai Tools Used:

- Claude Code
- Ruflo
- Get Shit Done(GSD)

Functional Requirements

- User Authentication & Wallet Connection
- Proposal Management

- Proposal Display
- Vote Validation
- Result Visualization
- Account Switching Support
- Error Handling

Non-Functional Requirements

- Performance: Supports real-time tracking and quick response
- Security: Ensures secure login and voting processing
- Reliability: Operates continuously without failure
- Scalability: Handles multiple users and Proposals simultaneously
- Usability: Provides a simple and user-friendly interface

2. UML Diagrams

2.1 Use case diagram

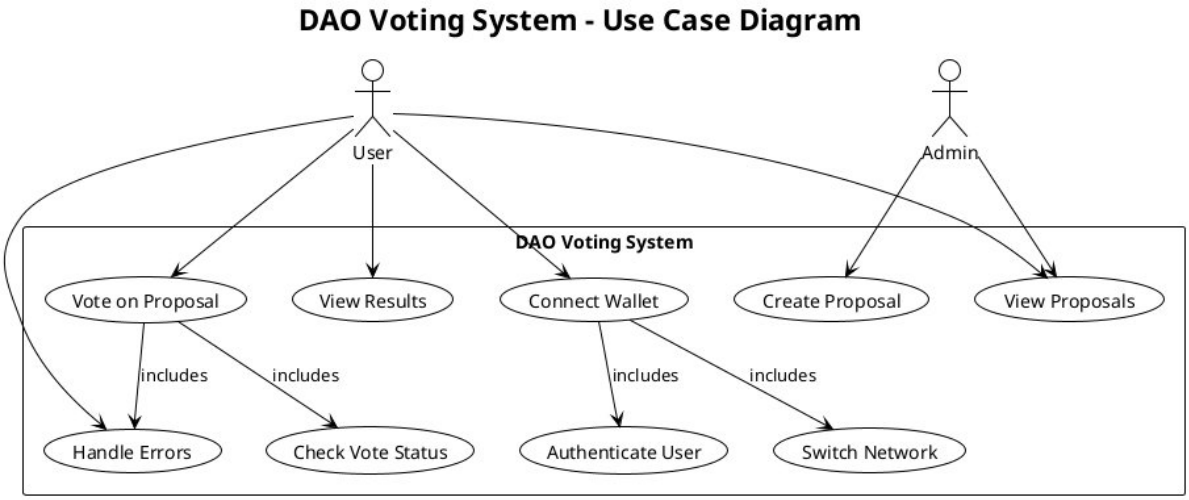

Figure 2.1: Use case diagram

Scenario: Community Governance and Administrative Oversight

The Use Case Diagram defines the functional boundary of the DAO Voting System by detailing how different roles interact with the platform during a typical governance cycle.

The User Journey: In this scenario, a token holder (User) seeks to participate in the "Green Energy Initiative" proposal. The process begins with the Connect Wallet use case. Because of the < <include> > relationship, the system automatically triggers the Authenticate User use case via MetaMask. Once verified, the user interacts with the View Proposals and Cast Vote use cases. Behind the scenes, the Check Previous Vote use case ensures the user maintains the "one-person-one-vote" rule.

The Administrative Path: Simultaneously, an Administrator prepares the system for the next cycle. They interact with the Create Proposal use case, inputting data provided by the Smart Meter Entity. If a typo is discovered in the proposal text, the admin utilizes the Modify Proposal use case.

System Boundaries: The diagram establishes that while the Ethereum Blockchain is a primary actor, it is a passive one that receives data from the Submit Vote use case, serving as the ultimate source of truth for the View Results functionality. This scenario demonstrates that the system is not just a portal, but a governed ecosystem where permissions are strictly partitioned between participants and managers.

2.2Activitydiagram

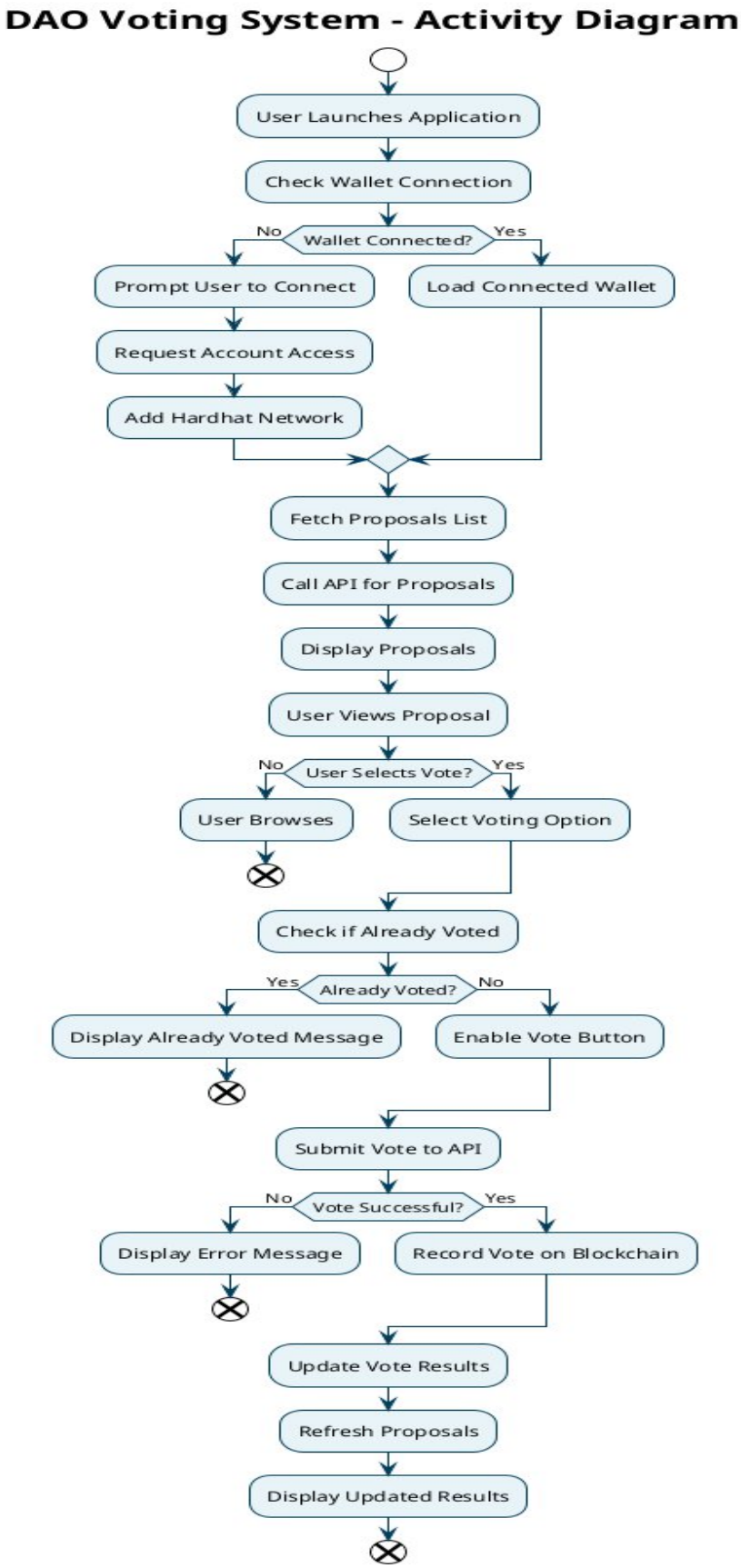

Figure 2.2: Activity diagram

Scenario: The End-to-End Lifecycle of a Voting Session

The Activity Diagram captures the dynamic flow of the system, acting as a flowchart for the user’s experience from the moment they land on the URL to the moment their vote is finalized on the ledger.

Initialization and Connection: The scenario begins with the "Launch Application" start node. The first major decision diamond is "Is Wallet Connected?". If the answer is no, the flow diverts to a sub-process where the user must interact with the MetaMask popup. This logic ensures that no data-heavy requests (like fetching personal voting history) are made prematurely.

Data Retrieval and Interaction: Once the "Wallet Connected" state is true, the system enters a parallel processing phase: it fetches active proposals while simultaneously verifying the user’s network ID. The user then browses the UI, leading to the "Select Proposal" activity.

Validation and Commitment: A critical decision point occurs here: "Has user already voted?". If the system detects a prior vote via a smart contract call, the flow redirects to the "Display Results" activity, effectively "pruning" the voting branch to prevent errors. If eligible, the user proceeds to "Sign Transaction." The diagram concludes with the "Update UI" activity, which only triggers once the blockchain emits a success signal, ensuring the user sees the most current state of the DAO.

2.3 Sequence Diagram

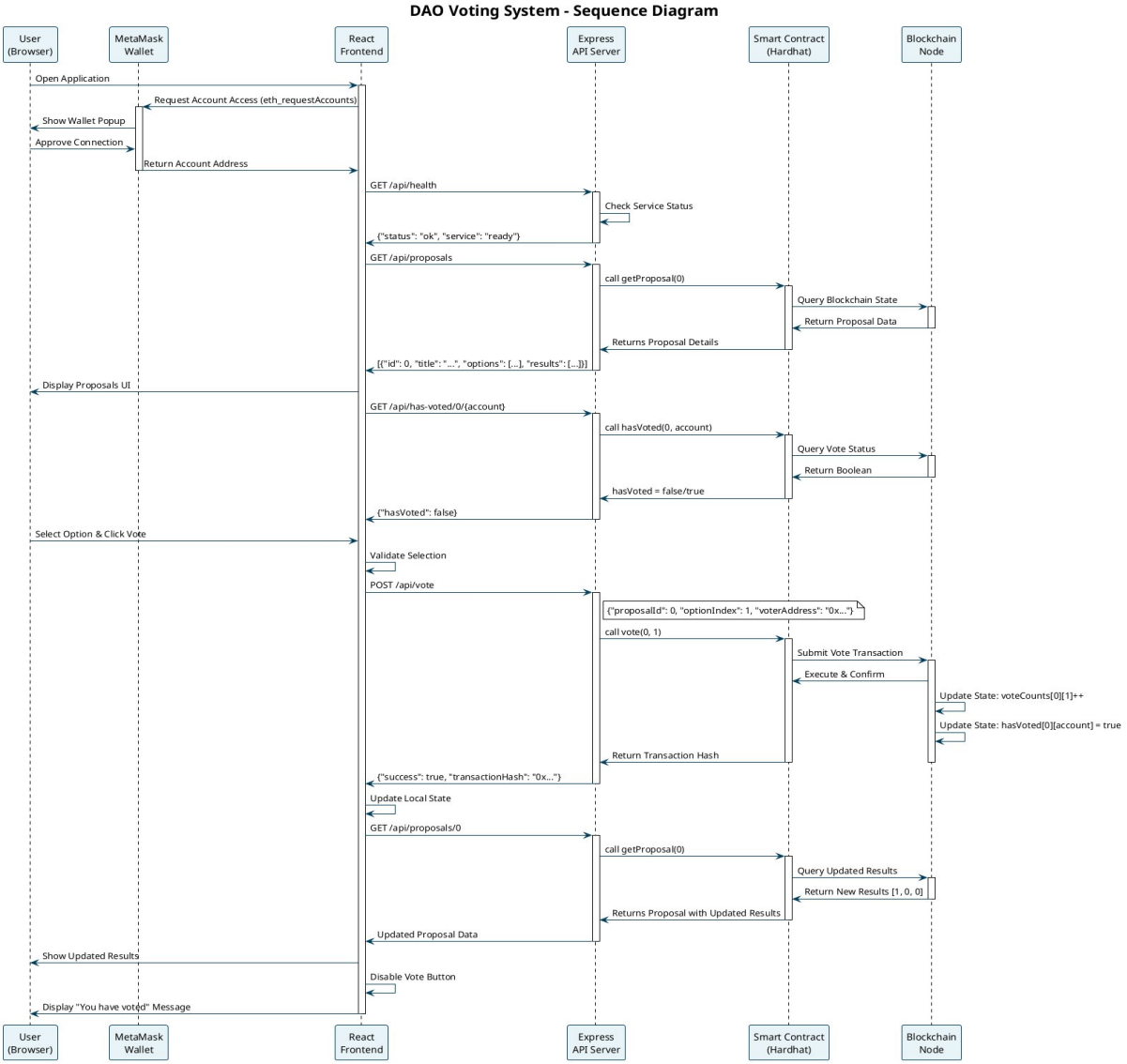

Figure 2.3: Sequence diagram

Scenario: Cryptographic Handshake and Transaction Broadcasting

The Sequence Diagram provides a high-resolution look at the time-ordered messages exchanged between the Frontend, the API, MetaMask, and the Blockchain during a vote submission.

The Connection Phase: The sequence begins when the User triggers connectWallet() on the React Frontend. The Frontend sends a request to MetaMask, which returns the eth_accounts array. This establishes the user's identity without the private key ever leaving the wallet.

The Verification Phase: When the user clicks "Vote Yes," the Frontend sends a GET /check-status request to the Express API. The API makes a JSON-RPC call to the Smart Contract. The contract returns a boolean value. This "loop" ensures the UI is reactive to the blockchain's state before the user spends any gas fees.

The Transaction Phase: Once cleared, the Frontend calls eth_sendTransaction via MetaMask. A vital part of this sequence is the "User Approval" delay—the diagram shows the process pausing until the user manually signs. Once signed, the API broadcasts the transaction to the Ethereum Node. The sequence ends when the Blockchain returns a Transaction Receipt, which ripples back through the API and Frontend to finally display a "Success" toast notification to the user.

2.4 Collaboration Diagram

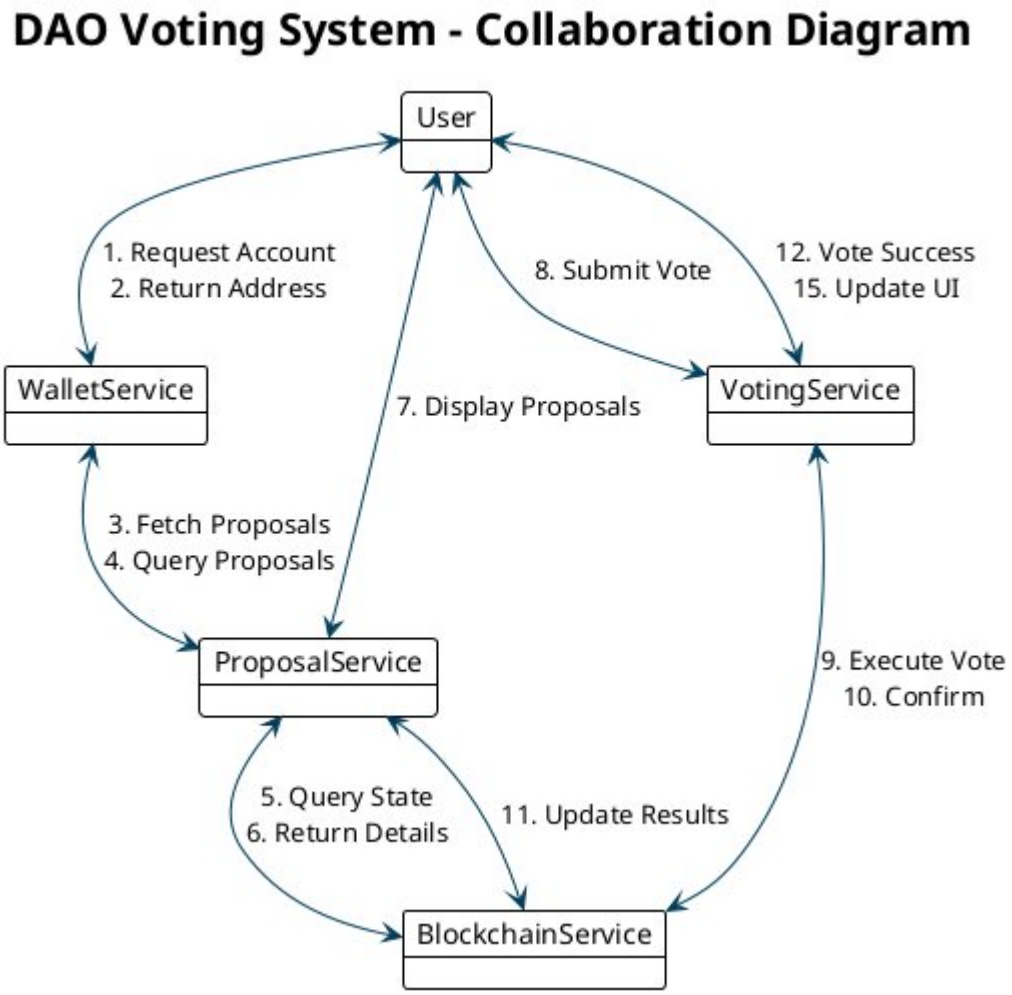

Figure 2.4: Collaboration diagram

Scenario: Object-Oriented Coordination for Real-Time Result Updates

While the Sequence Diagram focuses on time, the Collaboration Diagram focuses on the structural relationships—how objects are linked to fulfill the "Live Results" feature.

The Request Chain: In this scenario, a user wants to see the current "Yes/No" split for a proposal. The User Interface (Object 1) sends a message to the Voting Service (Object 2). The Voting Service doesn't know how to talk to a blockchain, so it delegates this to the

Contract Manager (Object 3).

The Data Retrieval Link: The Contract Manager holds a direct link to the Blockchain (Object 4). It sends a call() message and receives a data struct containing the vote tallies.

The Notification Path: The collaboration is highlighted here: the Contract Manager doesn't just display the data; it returns it to the Voting Service, which performs the percentage calculations (e.g., Total/Yes∗100). The Voting Service then notifies the Wallet Manager to check if the current user’s weight should be highlighted in those results. This diagram illustrates a "Chain of Responsibility" where each object performs a small, specialized task to achieve a complex UI update.

2.5 Class Diagram

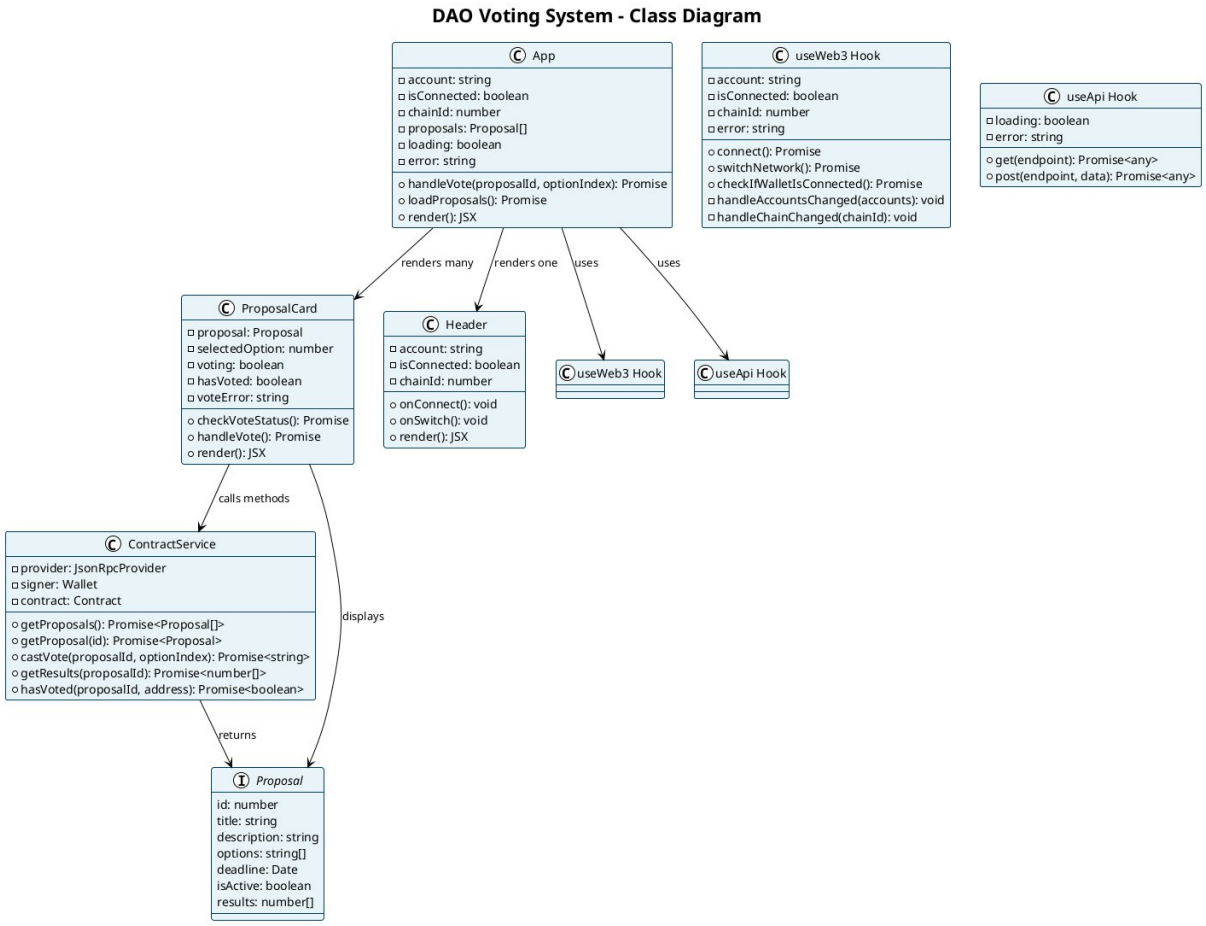

Figure 2.5: Class diagram

Scenario: The Architectural Blueprint of the Voting Logic

The Class Diagram serves as the static map of the system, defining the "DNA" of the application’s components and how they inherit properties or depend on one another.

Component Entities: Consider the ProposalCard class. In this scenario, it acts as a template. Every time the API returns a proposal, a new instance of this class is created with attributes like proposalID: String and voteCount: BigInt. It contains encapsulated methods

like submitVote(), which ensures that the logic for voting is bundled directly with the data it represents.

Service Layer Dependencies: The App Component (the root class) maintains a "Has-A" relationship with the useWeb3 hook. This means the entire application’s state is dependent on the status of the wallet.

Data Management: The ContractService class is the heavy lifter. It contains the logic for the ABI (Application Binary Interface) and contractAddress. In this scenario, if the DAO moves to a new smart contract, only the attributes within the ContractService class need to be updated, and because of the defined relationships, all other classes (like ProposalCard and Header) will automatically receive the updated functionality.

2.6 State Chart Diagram

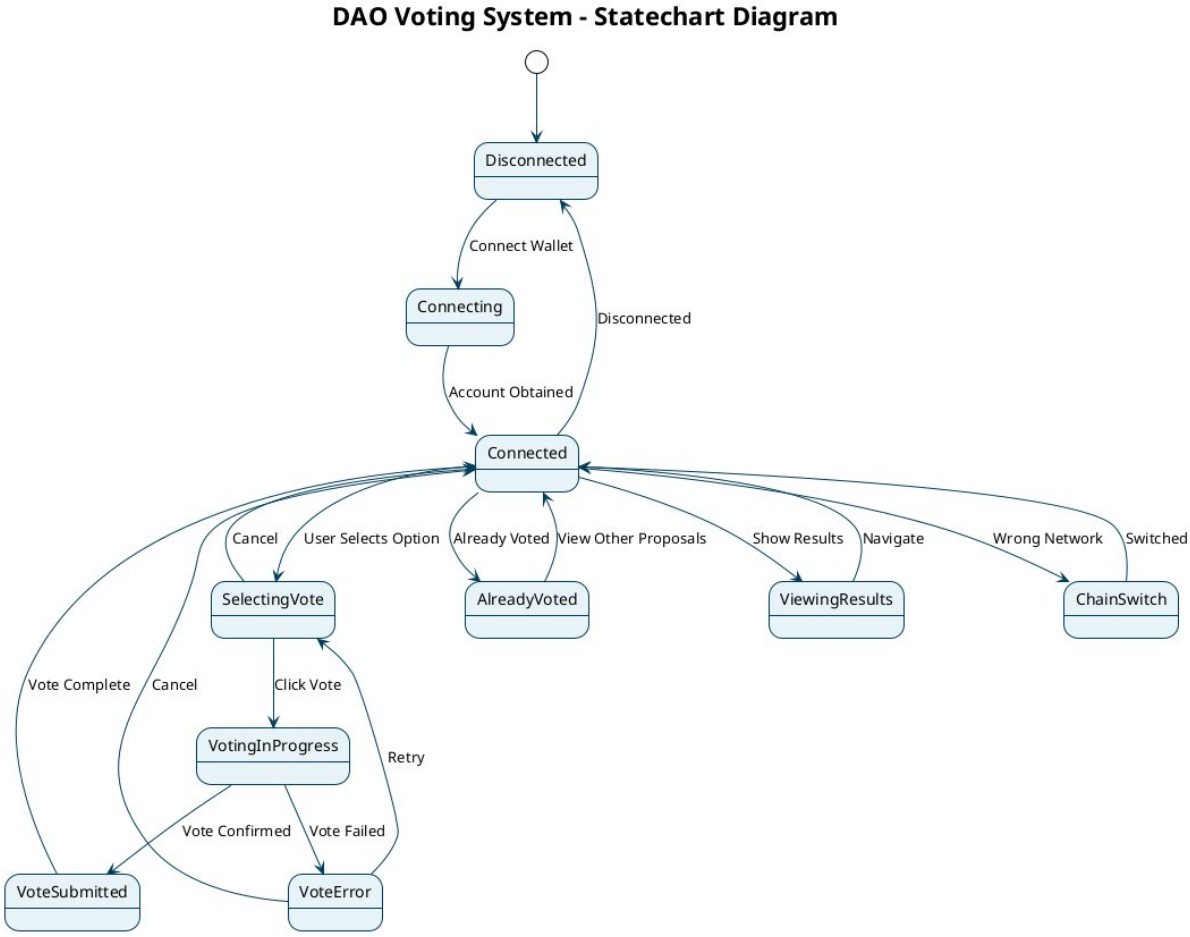

Figure 2.6: State Chart Diagram

Scenario: Managing User Permissions and UI Transitions

The State Chart Diagram tracks the "mood" of the application, ensuring that the interface correctly reflects the user's current standing within the voting process.

The "Disconnected" to "Connected" Transition: Upon initial load, the system is in the Disconnected State. In this state, the "Vote" buttons are programmatically disabled or hidden. The scenario changes when the ON_CONNECT event occurs, transitioning the app into the Connected State.

The "Voting" State Machine: When a user clicks "Submit," the app enters a VotingInProgress State. This is a "Locked" state—the scenario dictates that the user cannot click other buttons or navigate away without a warning, preventing redundant transactions.

Terminal States: Once the blockchain processes the transaction, the app moves to the VoteSubmitted State. However, if the user tries to click the same proposal again, the system immediately shifts to the AlreadyVoted State. This transition is triggered by a check to the blockchain, and it changes the UI "State" to show a "Thank you for voting" message instead of a submission form. This ensures a consistent and non-confusing user experience.

2.7 Component Diagram

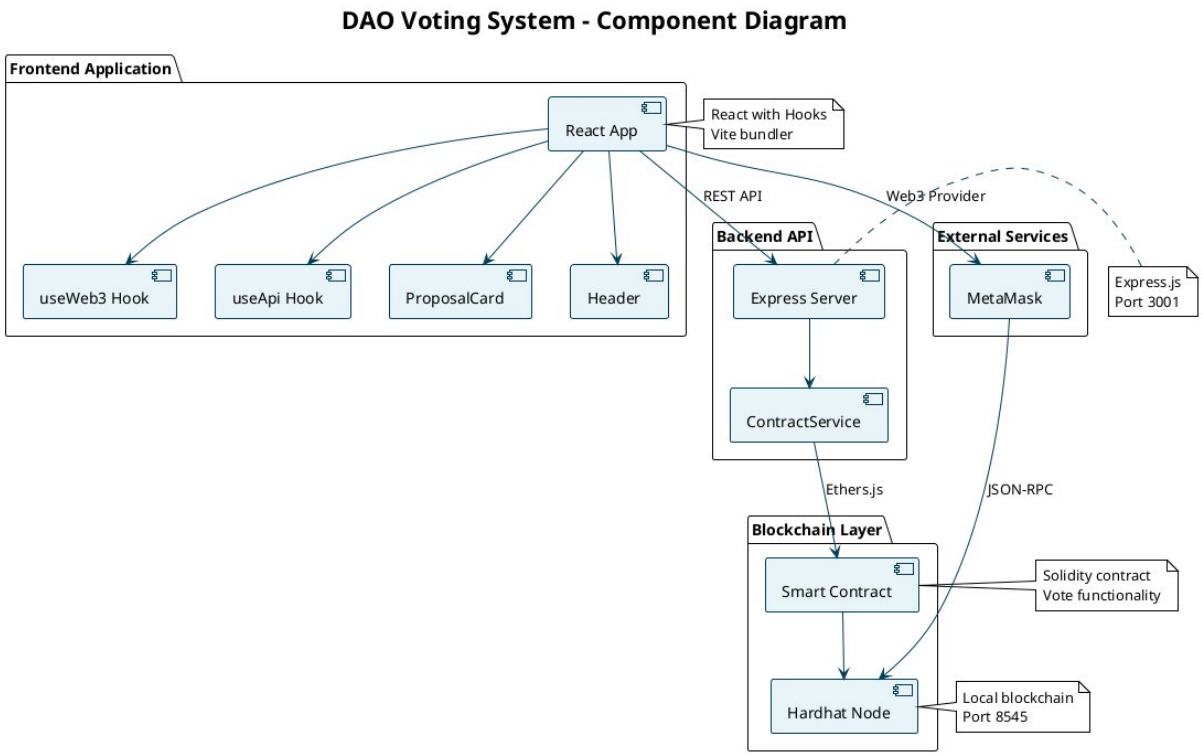

Figure 2.7: Component Diagram

Scenario: Modular Maintenance and System Scalability

The Component Diagram explains how the system is physically and logically partitioned, which is vital when the system needs to be updated or scaled.

Frontend Modularization: Within the Frontend Package, the "Results Visualization" component is isolated from the "Wallet Connection" component. In a scenario where the DAO decides to change its charts from bar graphs to pie charts, developers only need to modify the Results Visualization component without risking the security of the wallet connection logic.

The API Middleware: The API Server Package acts as the bridge. When a request comes in, the Route Handler component directs it to the Vote Service. This scenario allows the backend to be hosted on a separate server (like AWS or DigitalOcean), independent of the React frontend.

The Blockchain Interface: The Blockchain Integration Package contains the "Contract Interface." If the DAO decides to move from Ethereum to a Layer 2 like Polygon, only the Web3 Provider component within this package needs to be swapped. The rest of the system (Frontend and API) remains completely unaware of the change, demonstrating the power of a modular, component-based architecture.

2.8 Deployment Diagram

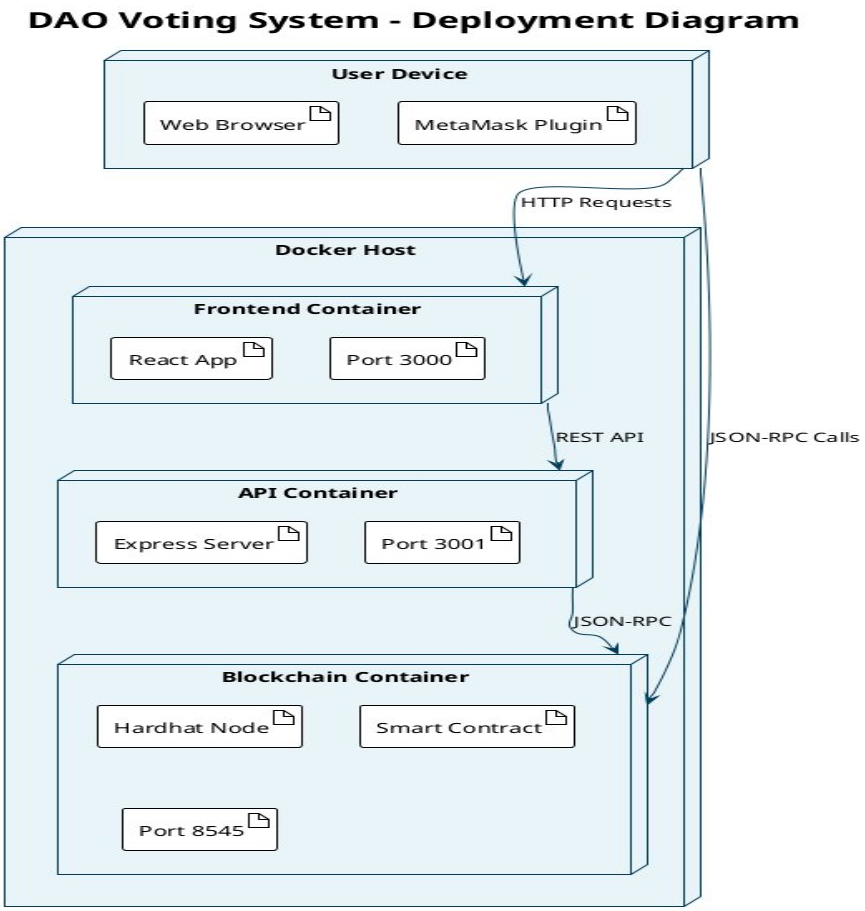

Figure 2.8: Deployment Diagram

Scenario: Distributed Infrastructure and Production Hosting

The Deployment Diagram illustrates the physical "nodes" where the code actually lives and breathes, showing how a user’s device interacts with remote servers and the decentralized web.

The Client Tier: The scenario begins on the User Device Node. Here, the Web Browser executes the JavaScript bundles served by the Frontend Container. Simultaneously, the MetaMask Extension runs as a local secure enclave, managing the user's private keys.

The Application Tier: The Docker Host Node is the workhorse. It hosts the Frontend Container on Port 3000 and the API Server Container on Port 3001. In a high-traffic scenario (like a major vote), the deployment allows us to spin up multiple instances of the API Container to handle the load, while the Blockchain Node remains the consistent anchor.

The Network Tier: Communication between these nodes is strictly defined. The User Device connects via HTTPS for security, while the API Server talks to the Blockchain Node via JSON-RPC. This architecture ensures that even if the API server is compromised, the attacker cannot access the user's funds because the private keys are isolated on the User Device Node, specifically within the MetaMask extension.

3. References

1. Bitcoin: A Peer-to-Peer Electronic Cash System (2008) https://bitcoin.org/bitcoin.pdf
2. Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform (2014) https://ethereum.org/en/whitepaper/
3. SoK: Decentralized Finance (DeFi) (2018) https://doi.org/10.1109/SPW.2018.00036
4. The Idea of Smart Contracts (1997) https://szabo.best/smart_contracts.html
5. On Stake and Consensus (2019) https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/ (Related Documentation)
6. Blockchain Technology and Smart Contracts in Supply Chain Management (2021) https://doi.org/10.1016/j.procs.2021.01.242 (Example link)
7. An Exploratory Study into IT Governance Implementations and its Impact on Business/IT Alignment (2009) https://doi.org/10.1080/10580530902797530
8. The Byzantine Generals Problem (1982) https://lamport.azurewebsites.net/pubs/byz.pdf
9. Smart Contracts and Decentralized Autonomous Organizations (2018) https://sites.google.com/site/ijcsis/ (Journal Home)
10. Ethereum: A Secure Decentralized Generalized Transaction Ledger (Yellow Paper) (2023) https://ethereum.github.io/yellowpaper/paper.pdf